

# Synchronous FIFO

## Design Specification

|             |                           |
|-------------|---------------------------|
| Document ID | FIFO-SPEC-001             |
| Version     | 1.0                       |
| Date        | 2026-05-05                |
| Status      | Released for Verification |
| Author      | RTL Design Team           |
| Reviewer    | Verification Team         |

■ **CONFIDENTIAL** This document contains proprietary RTL design information. Distribution restricted to the verification team.

## 1. Overview

This document describes the functional specification of a **synchronous, single-clock FIFO** (First-In First-Out) buffer intended for use as a data-rate decoupling element between producer and consumer logic residing on the same clock domain.

The FIFO operates in **First-Word-Fall-Through (FWFT)** mode: the oldest entry appears on `rd_data` combinatorially as soon as data is written, without requiring a dummy read-enable pulse from the consumer.

## 2. Parameters

| Parameter       | Default | Description                                                 |
|-----------------|---------|-------------------------------------------------------------|
| DATA_WIDTH      | 8       | Width of each data entry in bits                            |
| DEPTH           | 16      | Number of storage entries (must be power of 2)              |
| ALMOST_FULL_TH  | 14      | Fill level at/above which <code>almost_full</code> asserts  |
| ALMOST_EMPTY_TH | 2       | Fill level at/below which <code>almost_empty</code> asserts |

## 3. Port List

| Port                      | Dir | Width                    | Description                               |
|---------------------------|-----|--------------------------|-------------------------------------------|
| <code>clk</code>          | in  | 1                        | System clock — rising-edge triggered      |
| <code>rst_n</code>        | in  | 1                        | Synchronous active-low reset              |
| <code>wr_en</code>        | in  | 1                        | Write enable                              |
| <code>wr_data</code>      | in  | DATA_WIDTH               | Data word to write into FIFO              |
| <code>rd_en</code>        | in  | 1                        | Read enable — advances read pointer       |
| <code>rd_data</code>      | out | DATA_WIDTH               | Current head-of-FIFO data (combinatorial) |
| <code>full</code>         | out | 1                        | FIFO is completely full                   |
| <code>empty</code>        | out | 1                        | FIFO is completely empty                  |
| <code>almost_full</code>  | out | 1                        | Fill level $\geq$ ALMOST_FULL_TH          |
| <code>almost_empty</code> | out | 1                        | Fill level $\leq$ ALMOST_EMPTY_TH         |
| <code>overflow</code>     | out | 1                        | Sticky: write attempted when full         |
| <code>underflow</code>    | out | 1                        | Sticky: read attempted when empty         |
| <code>fill_level</code>   | out | $\log_2(\text{DEPTH})+1$ | Number of valid entries currently stored  |

## 4. Functional Description

4.1 Reset Behaviour

- `rst_n` is **synchronous** and active-low.
- On assertion: all pointers, count, flags, and `rd_data` reset to 0.
- The FIFO must be empty and all sticky flags cleared after reset de-asserts.

4.2 Write Operation

- A write occurs when `wr_en` = 1 AND `full` = 0 on the rising edge of `clk`.
- `wr_data` is stored at the current write-pointer location; the pointer then increments (wraps modulo DEPTH).
- If `wr_en` = 1 when `full` = 1, no write occurs and `overflow` latches HIGH.
- `overflow` is **sticky**: it remains HIGH until the next synchronous reset.

4.3 Read Operation — FWFT Mode

This FIFO implements **First-Word-Fall-Through (FWFT)**. The head of the FIFO is presented on `rd_data` **combinatorially** at all times when the FIFO is non-empty.

- `rd_en` = 1 advances the read pointer on the next rising clock edge.
- Data is valid on the **same cycle** that `rd_en` is asserted — no extra latency.
- If `rd_en` = 1 when `empty` = 1, no pointer changes and `underflow` latches HIGH.
- `underflow` is **sticky**: cleared only by synchronous reset.

4.4 Simultaneous Read and Write

A simultaneous write and read (`wr_en` = 1, `rd_en` = 1) is **legal at all fill levels**.

| Condition | Write | Read | Net Count | Flags        |
|-----------|-------|------|-----------|--------------|
| Normal    | ✓     | ✓    | stable    | No change    |
| Full      | ✓     | ✓    | stable    | No overflow  |
| Empty     | ✗     | ✗    | stable    | No underflow |

4.5 Status Flags

| Flag                      | Assertion Condition                           | Type          |
|---------------------------|-----------------------------------------------|---------------|
| <code>full</code>         | <code>fill_level == DEPTH</code>              | Combinatorial |
| <code>empty</code>        | <code>fill_level == 0</code>                  | Combinatorial |
| <code>almost_full</code>  | <code>fill_level &gt;= ALMOST_FULL_TH</code>  | Combinatorial |
| <code>almost_empty</code> | <code>fill_level &lt;= ALMOST_EMPTY_TH</code> | Combinatorial |
| <code>overflow</code>     | <code>wr_en &amp;&amp; full</code> (sticky)   | Registered    |
| <code>underflow</code>    | <code>rd_en &amp;&amp; empty</code> (sticky)  | Registered    |

4.6 Fill Level

- `fill_level` reflects the exact number of valid entries currently stored.
- Range: 0 (empty) to `DEPTH` (full). Width =  $\log_2(\text{DEPTH}) + 1$  bits.
- Must update on the same clock edge that a write or read completes.
- `fill_level == DEPTH` is equivalent to full; `fill_level == 0` is equivalent to empty.

## 5. Timing Diagrams

### 5.1 Basic Write then Read (FWFT)

After a write completes, `rd_data` reflects the written value **without** a preceding `rd_en` pulse.



### 5.2 Overflow (write when full)



## 6. Corner Cases — Mandatory Coverage

The verification plan **must** cover all of the following corner cases. Each must be hit at least once in directed tests and tracked in coverage.

| ID    | Description                                                                                  |
|-------|----------------------------------------------------------------------------------------------|
| CC-1  | Write to full FIFO — overflow flag asserts and latches                                       |
| CC-2  | Read from empty FIFO — underflow flag asserts and latches                                    |
| CC-3  | Simultaneous rd+wr when FIFO is empty                                                        |
| CC-4  | Simultaneous rd+wr when FIFO is full                                                         |
| CC-5  | Fill level exactly at <code>ALMOST_FULL_TH</code> boundary                                   |
| CC-6  | Fill level exactly at <code>ALMOST_EMPTY_TH</code> boundary                                  |
| CC-7  | Back-to-back writes until full, then back-to-back reads until empty                          |
| CC-8  | <code>rd_data</code> valid combinatorially without <code>rd_en</code> (FWFT check)           |
| CC-9  | Sticky overflow/underflow cleared only by reset, not by normal traffic                       |
| CC-10 | <code>fill_level</code> accuracy after full pointer wrap-around ( <code>DEPTH</code> writes) |
| CC-11 | Reset asserted mid-operation (write in progress)                                             |

|       |                                                        |
|-------|--------------------------------------------------------|
| CC-12 | Minimum depth: DEPTH=2 (smallest legal power-of-2 > 1) |
|-------|--------------------------------------------------------|

## 7. Assertion Requirements (SVA)

The testbench **must** include the following SystemVerilog Assertions. All assertions must pass for sign-off.

| #  | Assertion Description                                                     |
|----|---------------------------------------------------------------------------|
| A1 | full and empty must never be HIGH simultaneously                          |
| A2 | fill_level must equal the internal count register at all times            |
| A3 | overflow latches on write-when-full and never self-clears without reset   |
| A4 | underflow latches on read-when-empty and never self-clears without reset  |
| A5 | almost_full asserts whenever fill_level >= ALMOST_FULL_TH                 |
| A6 | almost_empty asserts whenever fill_level <= ALMOST_EMPTY_TH               |
| A7 | Data integrity: every byte written must be read back in strict FIFO order |
| A8 | No spurious pointer movement when wr_en=0 or rd_en=0                      |

## 8. Functional Coverage Goals

| Coverage Point                        | Target |
|---------------------------------------|--------|
| All flag transitions (0→1, 1→0)       | 100%   |
| fill_level values 0 through DEPTH     | 100%   |
| Simultaneous rd+wr at all fill levels | 100%   |
| Overflow condition hit                | Hit    |
| Underflow condition hit               | Hit    |
| Reset during active write             | Hit    |
| Reset during active read              | Hit    |
| almost_full exact threshold           | Hit    |
| almost_empty exact threshold          | Hit    |

## 9. Known Limitations / Out of Scope

- Asynchronous (dual-clock) FIFO is out of scope for this version.
- ECC or parity protection on stored data is not implemented.
- FIFO depth must be a power of 2; non-power-of-2 depths are undefined.

- Soft reset (non-synchronous) is not supported.

---

*End of Document — FIFO-SPEC-001 v1.0*